

Unitat 8.2

Subprogrames

Ús dels subprogrames

Com a qualsevol llenguatge de programació, són molt útils per:

- Modularitat: Permet dividir les funcionalitats en peces de codi més petites.
- Disseny simple: millora la llegibilitat del codi i oculta detalls.
- Mantenibilitat: Permet canviar detalls del codi canviant el mínim la seva estructura. Es pot canviar un subprograma sense tocar el codi que el crida.
- Reusabilitat: Un mateix subprograma es pot utilitzar des de diferents programes.

Tipus de subprogrames

Segons el seu funcionament:

- **Procediments (PROCEDURE)**

- Executen un codi.

```
PROCEDURE UPDATE_LLIBRE_EXEMPLARS(inllibre_id NUMBER, in_exemplars NUMBER);
```

- **Funcions (FUNCTION)**

- Executen un codi i retornen un valor, que ha de ser d'un tipus determinat.

```
FUNCTION QUEDEN_EXISTENCIES(inllibre_id NUMBER) RETURN BOOLEAN;
```

Tipus de subprogrames

Segons on s'emmagatzemen:

- NESTED: subprograma creat dins un bloc PL/SQL. **No es guarda a la base de dades.**
- PACKAGE: subprograma creat dins un paquet. **Es guarda a la base de dades** mentre el paquet hi sigui.
- STANDALONE: subprograma creat a nivell d'esquema. **Es guarda a la base de dades** mentre no s'elimini.

Parts d'un subprograma

- **Part declarativa** (opcional):
 - Declaracions de: tipus locals, variables, constants, cursors, excepcions i subprogrames niats. No comença amb DECLARE, si no amb IS.
- **Part executable** (obligatòria)
 - Conté les sentències del subprograma
- **Part de control d'excepcions** (opcionals)
 - Conté el codi per gestionar els errors que es puguin donar durant l'execució del subprograma.

Declaració d'un subprograma NESTED

Es declara a la part de declaració (DECLARE) del bloc PL/SQL:

```
DECLARE
```

```
    PROCEDURE nomProc(nomPar1 tipusPar1, nomPar2 tipusPar2, ...) IS
```

```
        --Part de declarativa del subprograma
```

```
    BEGIN
```

```
        --Part d'execució del subprograma
```

```
    EXCEPTION
```

```
        --Part de control d'errors del subprograma
```

```
    END;
```

```
BEGIN
```

```
    --Part d'execució del bloc anònim
```

```
END;
```

ALERTA! S'ha de declarar sempre després de les variables.

Declaració d'un subprograma NESTED

Si en lloc d'un procediment volguéssim declarar una funció:

```
DECLARE
    FUNCTION nomFunc(nomPar1 tipusPar1, nomPar2 tipusPar2, ...)
    RETURN tipusRetorn
    IS
        --Part de declarativa del subprograma
    BEGIN
        --Part d'execució del subprograma
    EXCEPTION
        --Part de control d'errors del subprograma
    END;
BEGIN
    --Part d'execució del bloc anònim
END;
```

ALERTA! S'ha de declarar sempre després de les variables.

Subprogrames NESTED: exemple

```
DECLARE
  PROCEDURE UPDATE_LLIBRE_EXEMPLARS(in_llibre_id NUMBER, in_exemplars NUMBER) IS
  BEGIN
    UPDATE LLIBRE SET EXEMPLARS = in_exemplars WHERE ID = in_llibre_id;
  END;

  FUNCTION QUEDEN_EXISTENCIES(in_llibre_id NUMBER) RETURN BOOLEAN IS
    vExemplars NUMBER;
  BEGIN
    SELECT exemplars INTO vExemplars FROM LLIBRE WHERE ID = in_llibre_id;
    RETURN NOT (vExemplars = 0);
  END;
BEGIN
  UPDATE_LLIBRE_EXEMPLARS(2,8);
  IF QUEDEN_EXISTENCIES(2) THEN
    DBMS_OUTPUT.PUT_LINE('Queden existències');
  ELSE
    DBMS_OUTPUT.PUT_LINE('No queden existències');
  END IF;
END;
```


Declaració d'un subprograma STANDALONE

S'ha d'utilitzar una sentència CREATE, fora de qualsevol bloc:

```
CREATE [OR REPLACE] PROCEDURE nomProc (par1 tipus, ...) IS
```

```
    --Part declarativa del subprograma
```

```
BEGIN
```

```
    --Part d'execució del subprograma
```

```
EXCEPTION
```

```
    --Part de control d'errors del subprograma
```

```
END;
```

Declaració d'un subprograma STANDALONE

Si és una funció:

```
CREATE [OR REPLACE] FUNCTION nomFunc (par1 tipus, ...)
RETURN tipusRetorn
IS
    --Part declarativa del subprograma
BEGIN
    --Part d'execució del subprograma
EXCEPTION
    --Part de control d'errors del subprograma
END;
```

Subprogrames STANDALONE: gestió

Es pot utilitzar DROP FUNCTION/PROCEDURE per eliminar subprogrames emmagatzemats en base de dades.

```
DROP PROCEDURE nomProcediment;
```

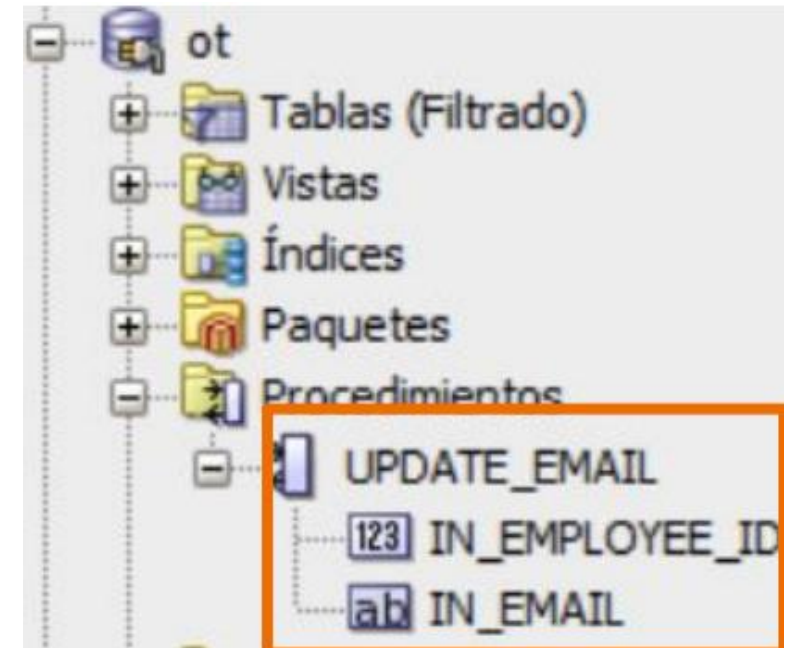
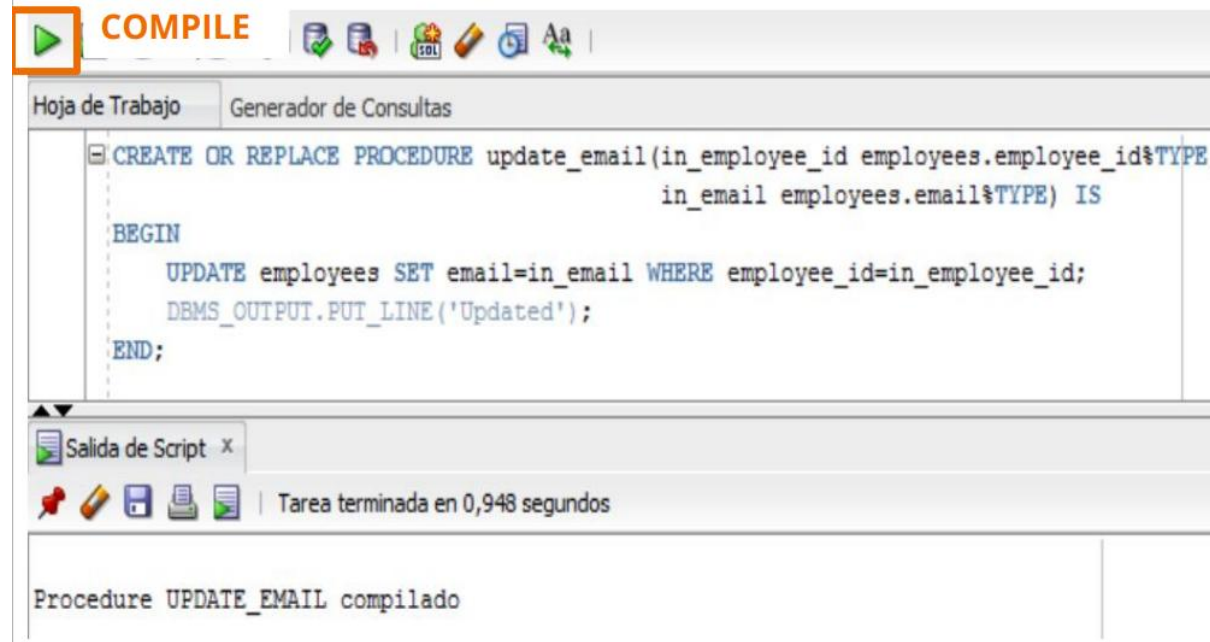
```
DROP FUNCTION nomFuncio;
```

També es poden controlar els permisos d'execució de subprogrames concrets:

```
GRANT EXECUTE ON nomFuncio TO nomUsuari;
```

```
REVOKE EXECUTE ON nomFuncio FROM nomUsuari;
```

A SQLDeveloper



Paràmetres del subprogrames

S'indiquen, entre parèntesi i separats per coma, després del nom del procediment o funció.

Síntaxi (per cada paràmetre):

`nom_parametre [mode] tipus`

Els tipus VARCHAR2 i NUMBER han d'anar SENSE mida.

~~VARCHAR2(50)~~ -> VARCHAR2, ~~NUMBER(3)~~ -> NUMBER

El mode és opcional i determina el comportament del paràmetre.

Modes dels paràmetres

Hi ha 3 modes de paràmetres:

- IN (per defecte): passa valors al subprograma.
 - Dins el subprograma, actua com una constant.
- OUT: Retorna un valor al codi que ha cridat al subprograma.
 - Dins el subprograma, actua com una variable.
 - Sempre s'ha de passar una variable com a paràmetre OUT.
- IN OUT: passa valors inicials al subprograma i en pot retornar d'actualitzats.
 - Sempre s'ha de passar una variable com a paràmetre IN OUT.

Modes dels paràmetres: exemple



```
DECLARE
    vExemplars LLIBRE.EXEMPLARS%TYPE;

    PROCEDURE UPDATE_LLIBRE_AFEGEIX_EXEMPLARS
        (in_llibre_id IN NUMBER, in_exemplars IN NUMBER, out_exemplars OUT NUMBER)
    IS BEGIN
        UPDATE LLIBRE SET EXEMPLARS = EXEMPLARS + in_exemplars WHERE ID = in_llibre_id;
        SELECT EXEMPLARS INTO out_exemplars FROM LLIBRE WHERE ID = in_llibre_id;
    END;
BEGIN
    UPDATE_LLIBRE_AFEGEIX_EXEMPLARS(2,4, vExemplars);
    DBMS_OUTPUT.PUT_LINE('Nous exemplars: '||vExemplars)
END;
```

Paràmetres amb valor per defecte

Per els paràmetres IN, es pot declarar un valor per defecte amb DEFAULT o amb “:=”.

```
DECLARE
    PROCEDURE UPDATE_LLIBRE_EXEMPLARS
        (inllibre_id NUMBER DEFAULT 2,
         in_exemplars NUMBER DEFAULT 0)
    IS BEGIN
        UPDATE LLIBRE
            SET EXEMPLARS = in_exemplars
            WHERE ID = inllibre_id;
    END;
BEGIN
    UPDATE_LLIBRE_EXEMPLARS( );
END;
```

```
DECLARE
    PROCEDURE UPDATE_LLIBRE_EXEMPLARS
        (inllibre_id NUMBER := 2,
         in_exemplars NUMBER := 0)
    IS BEGIN
        UPDATE LLIBRE
            SET EXEMPLARS = in_exemplars
            WHERE ID = inllibre_id;
    END;
BEGIN
    UPDATE_LLIBRE_EXEMPLARS( );
END;
```


Execució de subprogrames NESTED

```
DECLARE
    v_concatResult varchar2(255);
    PROCEDURE PRINT(MSG VARCHAR2 := 'Praise the Sun!') IS BEGIN
        DBMS_OUTPUT.PUT_LINE(MSG);
    END;
    FUNCTION CONCAT(MSG VARCHAR2, MSG2 VARCHAR2) RETURN VARCHAR2 IS BEGIN
        RETURN MSG||MSG2;
    END;
BEGIN
    PRINT;
    PRINT( );
    PRINT( 'Hola' );

    v_concatResult := CONCAT('A', 'B');
    SELECT CONCAT('A', 'C') INTO v_concatResult;
    PRINT(CONCAT('Hola', ' i adéu'));
END;
```

Execució de subprogrames STANDALONE

2 alternatives:

- Des de dins un bloc directament amb el nom del subprograma.
- Utilitzant EXECUTE.

```
create or replace PROCEDURE PRINT
    (MSG VARCHAR2 := 'NULL')
IS
BEGIN
    DBMS_OUTPUT.PUT_LINE(MSG);
END;
```

```
EXECUTE PRINT('Hola');

BEGIN
    PRINT;
END;
```

Subprogrames: exercici

Sobre HR

1. Declara una funció que retorni el job_title d'un empleat (EMPLOYEES), donat el seu FIRST_NAME i LAST_NAME.
2. Declara un procediment que “retorni” el correu electronic i el telèfon d'un empleat, donat el seu FIRST_NAME i LAST_NAME. Utilitza paràmetres OUT.
3. Utilitza els dos subprogrames anteriors a un bloc anònim per obtenir el job_title, el correu i el telèfon de l'empleat “Blake Cooper”.